
Distributed Systems

Monsoon 2025

Lecture 8 (b)

International Institute of Information Technology

Hyderabad, India

What is a FileSystem?

- Provides support for access and also access control.
- The Unix/Linux file system has a mechanism based on inodes.
 - Direct and indirect numbering
 - Access control based on owner/ group/ others, for read/ write/ execute.
 - Formed the basis for many other file system designs.
- For more details, we refer the reader to the book by Bach.

Read a few chapters of the book by Maurice J Bach on the Design of the Unix Operating System

What is a File System?

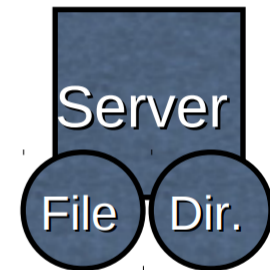
Client

File Ops

Open
Read
Write
Close
Seek

Directory Ops

Create file
Mkdir
Rename file
Rename directory
Delete file
Delete directory



Current Trends

- Emergence of planet-scale applications such as Facebook and YouTube.
- These applications allow multiple users to create, share, and access content simultaneously.
- The amount of content generated each day is of the order of petabytes or more.
- Most of the data and the users of the data are more likely to be geographically separated yet seamlessly connected.
- All data is not the same!
- Data is the new oil – source of analytics, revenue, systems, ...

Current Trends

- To benefit from the data, this data has to be stored in appropriate manner, including crossing hurdles such as
 - Scalability
 - Retrieval
 - Network latency
 - Concurrent access
 - Availability
 - Durability
 - And so on

Design Features for Distributed File Systems

- Durability
 - The ability to recover data in case of failures to the underlying hardware.
 - Distributed file systems usually employ a form of replication to safeguard against such issues.
 - Replication however comes with a set of challenges involving the creation/ updation/ maintenance/ and consistency of the replicas.

Design Features for Distributed File Systems

- Availability
 - Distributed file systems have to contend with possible errors from the users
 - Errors could come from the storage system too.
 - Think of multiple disk drives or other devices that have many mechanisms prone to failure over time.
 - Delays and other network induced vagaries affect the file system and its operations

Design Features for Distributed File Systems

- Consistency Model
 - Multiple users create/ read/ update/ write to the same set of files, possibly at the same time.
 - Think of systems such as git, dropbox, google docs, OneDrive etc.
 - Need mechanisms to support such concurrent updates
 - Semantics and guarantees on what happens to concurrent updates

Design Features for Distributed File Systems

- Scalability
 - Provide for scale as the storage requirements grow
 - Allow for scaling up the capacity in a seamless and transparent manner without degrading performance.

Design Features for Distributed File Systems

- Metadata
 - Namespace and the associated data structures form the bulk of how a file system maintains files.
 - What is the Unix model for namespace and metadata?
 - Understand the memory system considerations while using the data structures of choice
 - These choices impact latency and throughput.
 - How?

Design Features for Distributed File Systems

- Other Application Specific Concerns
 - What are the typical use cases?
 - Helps understand system features and requirements
 - What are the typical file sizes?
 - Helps decide on parameters such as block size.
 - What functionality or functionalities are more important from a performance point of view?
 - To support better throughput, may want to optimize on key functionalities even at the expense of some others.

The Design Space

- Now that we have identified a few issues, let us understand the potential choices for system designers.

The Design Space

- Flat vs Hierarchical namespace
 - Flat namespace requires each file to be given a unique identifier.
 - These identifiers are used to locate the required file.
 - Hierarchical namespace allows for a pathname based identification.

The Design Space

- How the read/write are supported?
- Some of the options are
 - A fully client-server system
 - Server hands off to client.
 - Server helps client locate the file
 - Server allows the client to cache the file locally either in full or in parts.
 - Can create inconsistent views.

The Design Space

- Guarantees on consistency has a full spectrum of possibilities.
 - Leave to the clients: no guarantees offered.
 - Forbid concurrent updates. Not very useful
 - Last Write Wins: Serialized view of writes, and the last successful write is what the server keeps
 - What do some of the systems you know do?

The Design Space

- Application specific concerns
 - Combine multiple objects into a single file
 - Photos in facebook
 - Disallow write/update and create a new copy on update.
 - Support operations such as append instead of write.
 - Log files

The Design Space

- Handling faults
 - Provide for multiple servers
 - Replication of data and its associated challenges

Distributed Systems

Monsoon 2025

Lecture 9

International Institute of Information Technology

Hyderabad, India

Distributed File Systems

- Earliest versions of distributed file systems:
 - **NFS** – Networked File System
 - Developed at Sun Microsystems which believed in the slogan “The Network is the Computer”
 - **AFS** – Andrew File System
 - Part of the Andrew project at CMU that created a distributed computing environment at CMU.
- Modern projects on distributed file systems include
 - **GFS** – The Google File System,
 - **HDFS** – the OpenSource version of GFS
 - **Colossus** – the next version of GFS
 - **Haystack, Tectonic** – the systems used by Facebook
- Among all, there are some common notions that we will explore today.

Goals of NFS and AFS

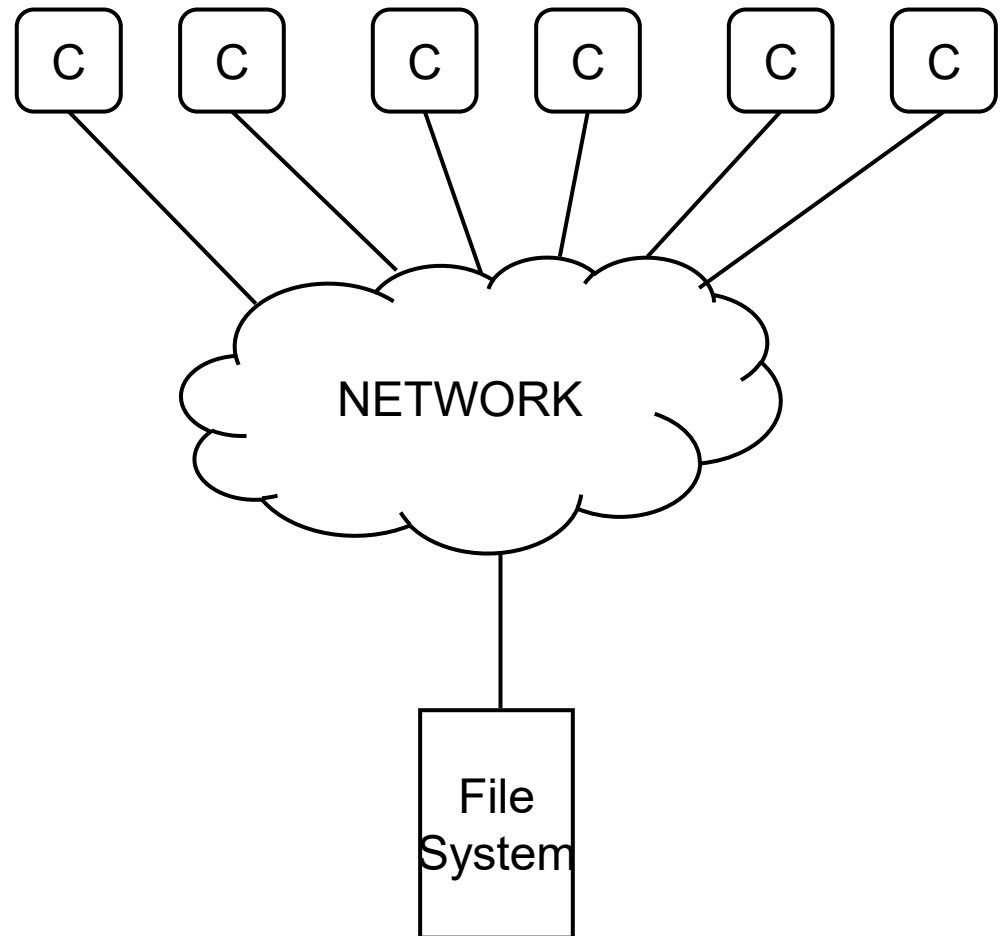
- Typical goals of a file system:
 - Have a consistent namespace for files
 - Let authorized users access their files and perform file operations
- On the other hand, distributed systems and distributed file systems have a variety of goals including
 - **Scalability:** Support large user base and large file base
 - **Fault tolerance:** Tolerate client and server failures
 - **Concurrency:** Allow multiple users
 - **Security:** Authorized users only
 - ...
- **Cannot meet all goals all the time**
 - Prioritize towards most important goals.

Design Principle of Distributed File Systems

- **User-Oriented Vs. Workload-oriented design**
 - Workload oriented design measures the characteristics of target workloads to inform the design
 - On the other hand, user-oriented designs optimize the system towards how users use files.
- **User-oriented designs** optimize to how users use files (vs. big programs)
 - Most files are privately owned
 - Not too much concurrent access
 - Sequential access is common; reads more common than writes
 - Example: AFS and NFS
- Distributed file systems such as GFS are geared towards big-program/**big-data workloads**

Early Design Options – NFS

- Use **RPC** to forward every FS operation to the server
 - Server orders all accesses, performs them, and sends back result
 - NFS v1 was built like this.



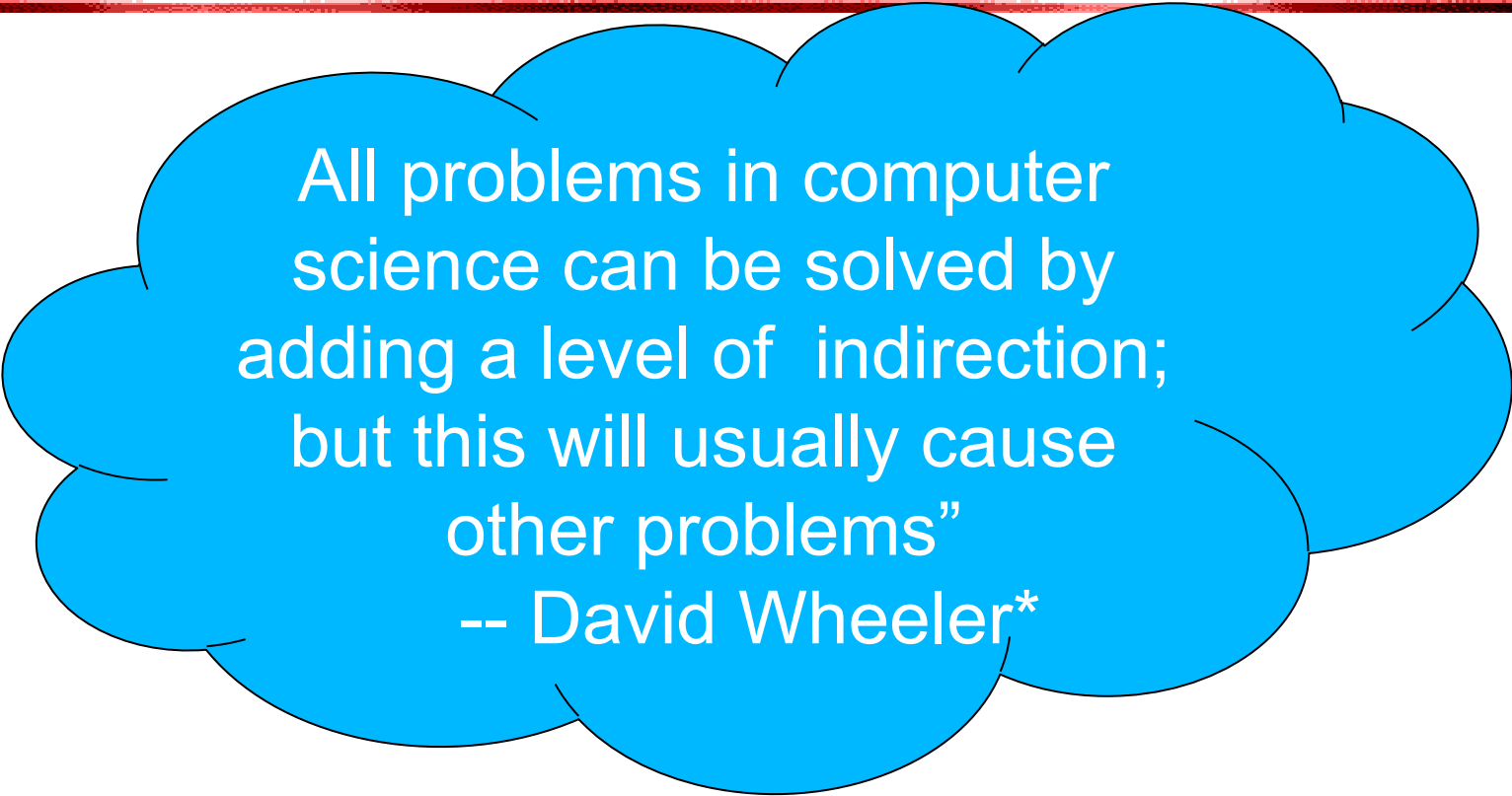
Early Design Options

- Use **RPC** to forward every FS operation to the server
 - Server orders all accesses, performs them, and sends back result
 - NFS v1 was built like this.
- **Plus:** Same behavior as if both programs were running on the same local filesystem!
- **Minuses:**
 - Performance will suffer. Latency of access to remote server often much higher than to local memory.
 - Moreover, server would get overloaded as accessing server is needed for every action.

Early Design Options

- Use **RPC** to forward every FS operation to the server
 - Server orders all accesses, performs them, and sends back result
 - NFS v1 was built like this.
- **Plus:** Same behavior as if all programs were running on the same local filesystem!
- **Minuses:**
 - Performance will suffer. Latency of access to remote server often much higher than to local memory.
 - Moreover server would get overloaded as accessing server is needed for every action.
- **Moral:** We need designs that avoid accessing the server for everything? What can we avoid this for? What do we lose in the process?

A Better solution



All problems in computer science can be solved by adding a level of indirection; but this will usually cause other problems”
-- David Wheeler*

- Per above, we will try the same idea for distributed file systems along with **caching**.
- But we should understand the risks of **caching** in terms of **consistency**.

A Better solution

All problems in computer science can be solved by adding a level of indirection; but this will usually cause other problems”
-- David Wheeler*

- Per above, we will try the same idea for distributed file systems along with **caching**.
- But we should understand the risks of **caching** in terms of **consistency**.

* of the Burrows-Wheeler Transform used in data compression

Problems with Caching

- **Server crashes**
 - What if client does
 - `seek(); /* SERVER CRASH */; read()`
 - If server maintains file position in RAM, the read will return bogus data
 - Any data that's in memory but not on disk is lost
- **Lost messages**
 - What if a client loses an acknowledgement from the server for `delete("foo")`
 - And in the meantime, another client created the file with the same name anew?
 - The first client might retry the delete and delete new file
- **Client crashes**
 - Might lose data updates in client cache

Stateful Vs Stateless Protocols

- **Stateful Protocols:** Server maintains client-specific state
 - Shorter requests
 - Better performance in processing requests
 - Cache coherence is possible – Server can know who's accessing what
 - File locking is possible
- **Stateless Protocols:** Server maintains no information on client accesses
 - Server can crash and recover – No state to lose
 - Client can crash and recover
 - Each request must identify file and offsets
 - No open/close needed
 - They only establish state
 - No server space used for state – can support many clients
 - File locking not possible

NFS Solution

- **Stateless design**
 - Flush-on-close: When file is closed, all modified blocks sent to server. close() does not return until bytes safely stored.
- Stateless protocol: **requests specify exact state.**
 - read() → read([position]). no seek on server.
- Operations are **idempotent**
 - How can we ensure this? Unique IDs on files/directories.
 - It's not delete("foo"), it's delete(1337f00f), where that ID won't be reused.
 - See the level of **indirection**

NFS

- Cache file blocks, directory metadata in RAM at both clients and servers.
- **Plus**: No network traffic if open/read/write/close can be done locally.
- **Minus**: failures and consistency are big concerns with this approach
- NFS trades some consistency for increased performance...

Caching and lack of Consistency

- A writer and a reader may notice inconsistency due to writes not taking effect at the server but only in the client.
- NFS **allows** for this inconsistency.
- Requires **flush on close**.
 - Flush all updates from cache to server on close operation.
- This means the **system can be inconsistent** for a while
 - Two clients doing a read() at the same time for the same file could see different results if one had old data cached and the other didn't.
- Called as **weak consistency**.
- Periodic checks to minimize damage.
- NFS provides **no guarantees at all on multiple writes**.

Later Versions of NFS

- Beyond NFS v1.0, there were NFS versions up to NFS v4.0 developed over the years.
- NFS v3.0 introduced the **asynchronous write** operation.
 - In the semantics of asynchronous write operation, the client sends the write data to the server and the server acknowledges the receipt of the data.
 - However, the server need not write the data to stable storage before sending the reply.
 - Asynchronous writes provide the server with several options to determine the best policy to synchronize the data.
 - The server may never schedule the write or may wait for multiple writes to be performed together

Later Versions of NFS

- NFS v3.0 introduced the asynchronous write operation.
- NFS v3.0 introduced basic file locking also via the Network Lock Manager.
 - This now becomes a stateful protocol.
- NFS v4.0 is a stateful protocol and adds support for
 - **Compound operations**: combine a sequence of operations to a single remote operation.
 - Other additional features: **locking** as a first class feature, **leases**.

AFS in Brief

- With some commonalities, we will study about AFS in brief.
 - Read in detail offline.
- AFS includes
 - More aggressive caching
 - Prefetching (on open, AFS gets entire file from server, making subsequent ops local & fast)
 - Close-to-open consistency only
 - All changes flushed to the server on close
 - Why does this make sense? (user-centric workloads)
 - Cache **invalidation** callbacks
 - Clients register with server that they have a copy of file
 - Server tells them: “Invalidate!” if the file changes
 - This trades server-side state for improved consistency

Summary of NFS and AFS

- For both AFS and NFS, **central server** is:
 - **Bottleneck**: reads / writes hit it at least once per file use
 - **Single point of failure**
 - **Expensive**: to make server fast and reliable, you need to go for expensive hardware
- GFS addresses some of these concerns
 - But tailored for slightly different users/workloads.

Overview of GFS

- Design goals/priorities
 - Design for big-data workloads
 - Huge files, mostly appends, concurrency, huge bandwidth
 - Design for failures
- Interface: non-POSIX
 - New op: record appends (atomicity matters, order doesn't)
- Architecture: one master, many chunk (data) servers
 - Master stores metadata, and monitors chunk servers
 - Chunk servers store and serve chunks
- Semantics
 - Nothing for traditional write operation
 - At least once and atomic for record appends

GFS Workload Characteristics

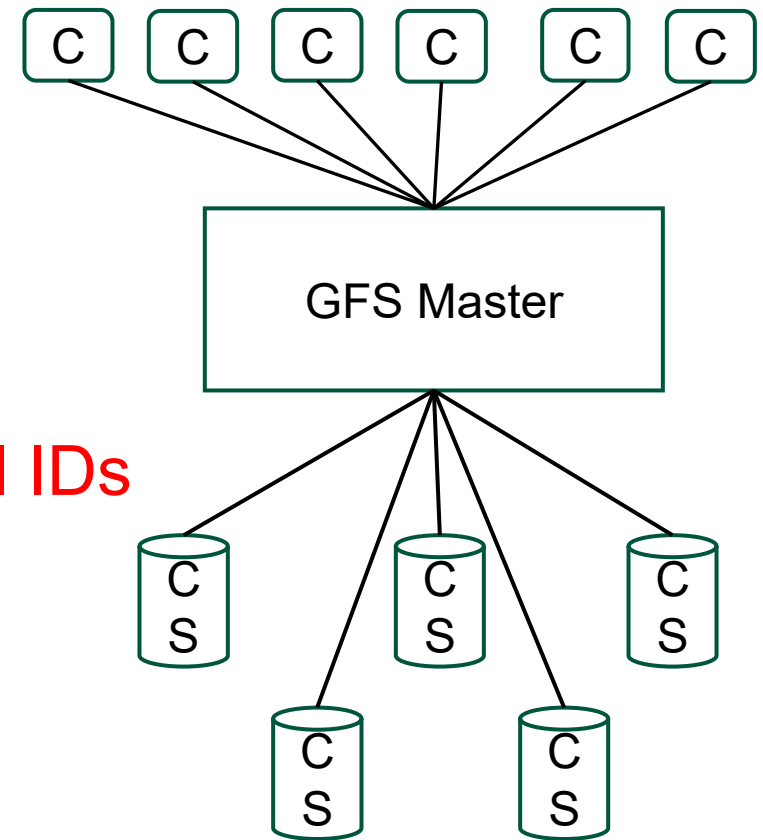
- Files are huge by traditional standards
 - Multi-GB files are common
- Most file updates are appends
 - Random writes are practically nonexistent
 - Many files are written once, and read sequentially
- Lots of concurrent data accessing
 - E.g., multiple crawler workers updating an index file

GFS Interface

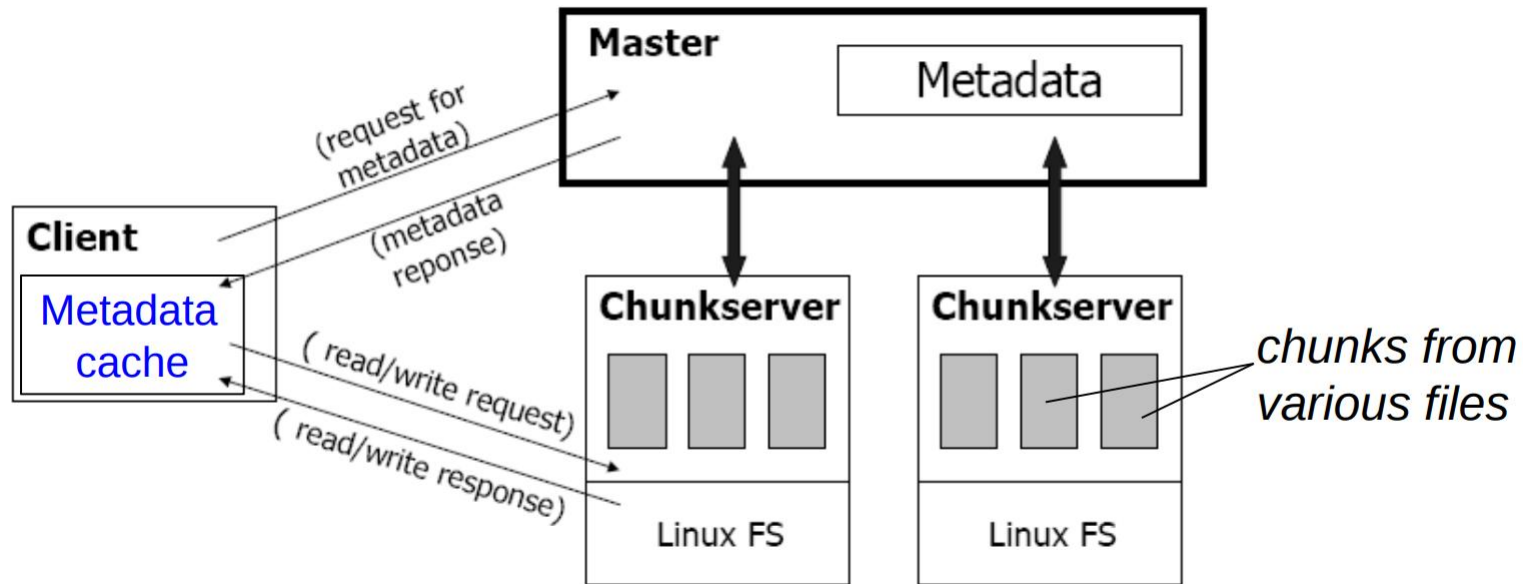
- Not POSIX compliant
 - Supports only popular FS operations, and semantics are different
 - That means you wouldn't be able to mount it
- Additional operation: **record append**
 - Frequent operation
 - Merging results from multiple machines in one file (Map/Reduce)
 - Using file as a producer - consumer queue
 - Logging user activity, site traffic
 - **Order doesn't matter for appends, but atomicity and concurrency matters**

GFS Architecture

- A GFS cluster
 - A **single master** (replicated later)
 - **Many** chunkservers
 - Accessed by many clients
- A file
 - Divided into fixed-sized **chunks** (similar to FS blocks)
 - Labeled with **64-bit unique global IDs** (called handles)
 - Stored at **chunkservers**
 - **3-way replicated** across chunkservers
 - **Master keeps track of metadata** (e.g., which chunks belong to which files)

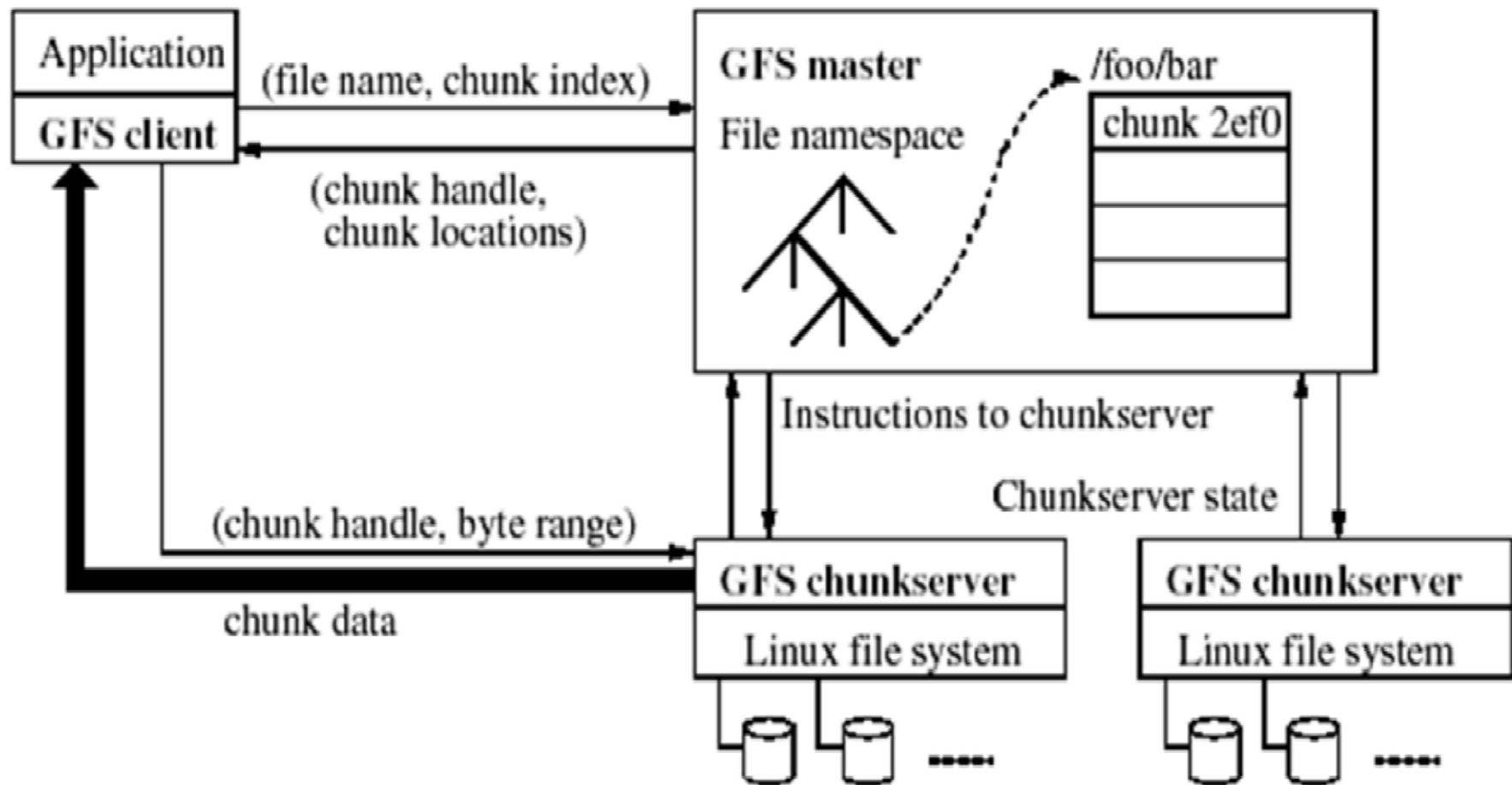


GFS Basic Functioning



- **Client retrieves metadata** for operation from master
- Read/Write **data flows between client and chunkserver**
- Minimizing the master's involvement in read/write operations alleviates the single-master bottleneck

Architecture in Picture



Legend:



Data messages



Control messages

Chunks

- **Larger chunks:** Analogous to FS blocks, except larger
 - Size: 64 MB!
 - Normal FS block sizes are 512B - 8KB
- **Pros of big chunk sizes:**
 - Less load on server (less metadata, hence can be kept in master's memory)
 - Suitable for big-data applications (e.g., search)
 - Sustains large bandwidth, reduces network overhead
- **Cons of big chunk sizes:**
 - Fragmentation if small files are more frequent than initially believed

The GFS Master

- **A process** running on a separate machine
 - Initially, GFS supported just a single master, but then they added master replication for fault-tolerance in other versions/distributed storage systems
- **Stores all metadata**
 - File and chunk namespaces
 - Hierarchical namespace for files, flat namespace for chunks
 - File-to-chunk mappings
 - Locations of a chunk's replicas

Chunk Locations

- Kept in memory, no persistent states
 - Master polls chunkservers at startup
- What does this imply?
 - Upsides: master can restart and recover chunks from chunkservers
 - Note that the hierarchical file namespace is kept on durable storage in the master
 - Downside: restarting master takes a long time
- Why do you think they do it this way?
 - Design for failures
 - Simplicity
 - Scalability – the less persistent state master maintains, the better

GFS Master $\leftrightarrow$ Chunkservers

- Master and chunkserver communicate regularly (**heartbeat**):
 - Is a chunkserver down?
 - Are there disk failures on a chunkserver?
 - Are any replicas corrupted?
 - Which chunks does a chunkserver store?
- Master sends **instructions** to chunkserver:
 - Delete a chunk
 - Create a new chunk
 - Replicate and start serving this chunk (chunk migration)
 - Why do we need migration support?

Distributed Systems

Monsoon 2025

Lecture 10

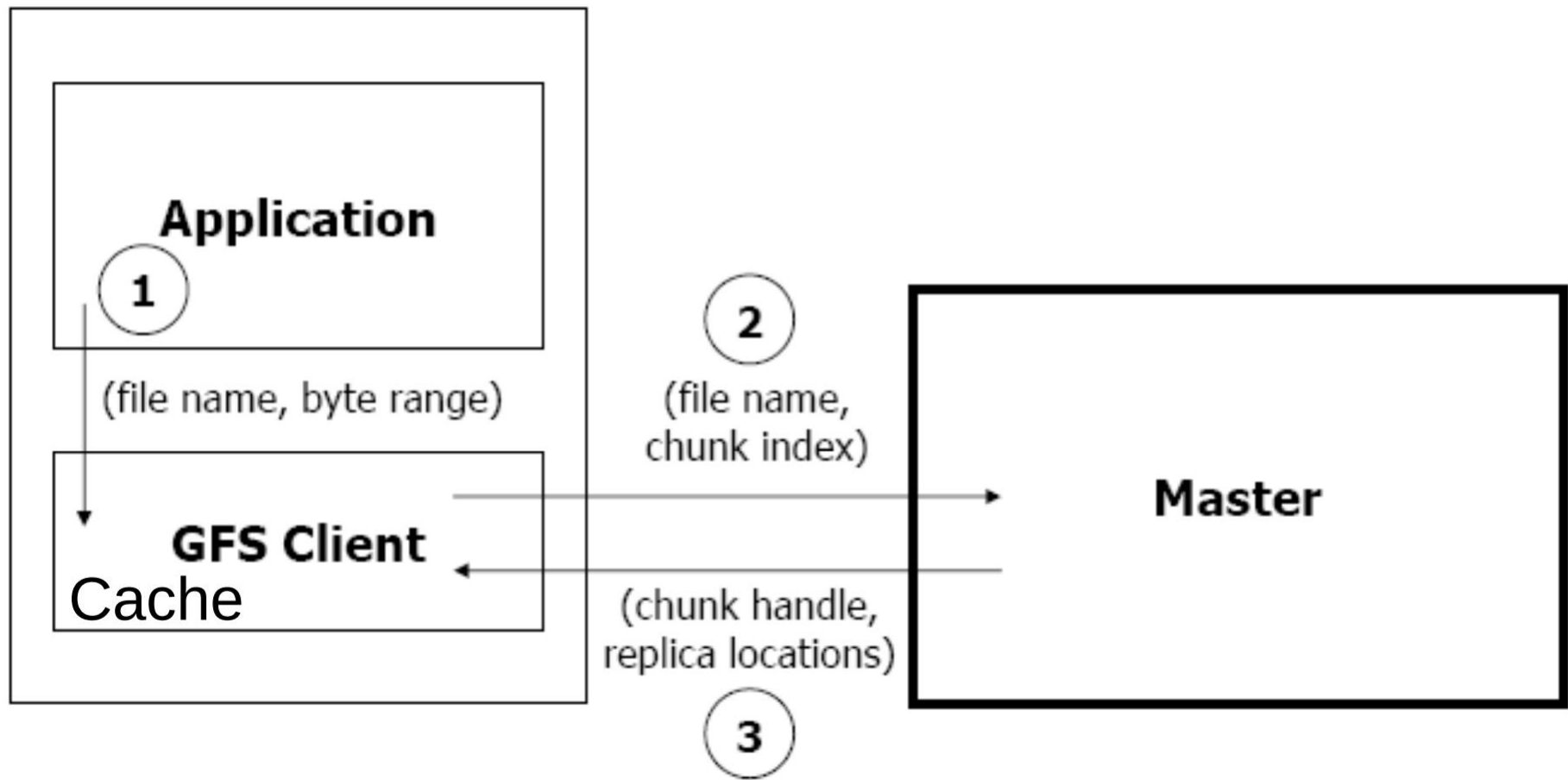
International Institute of Information Technology

Hyderabad, India

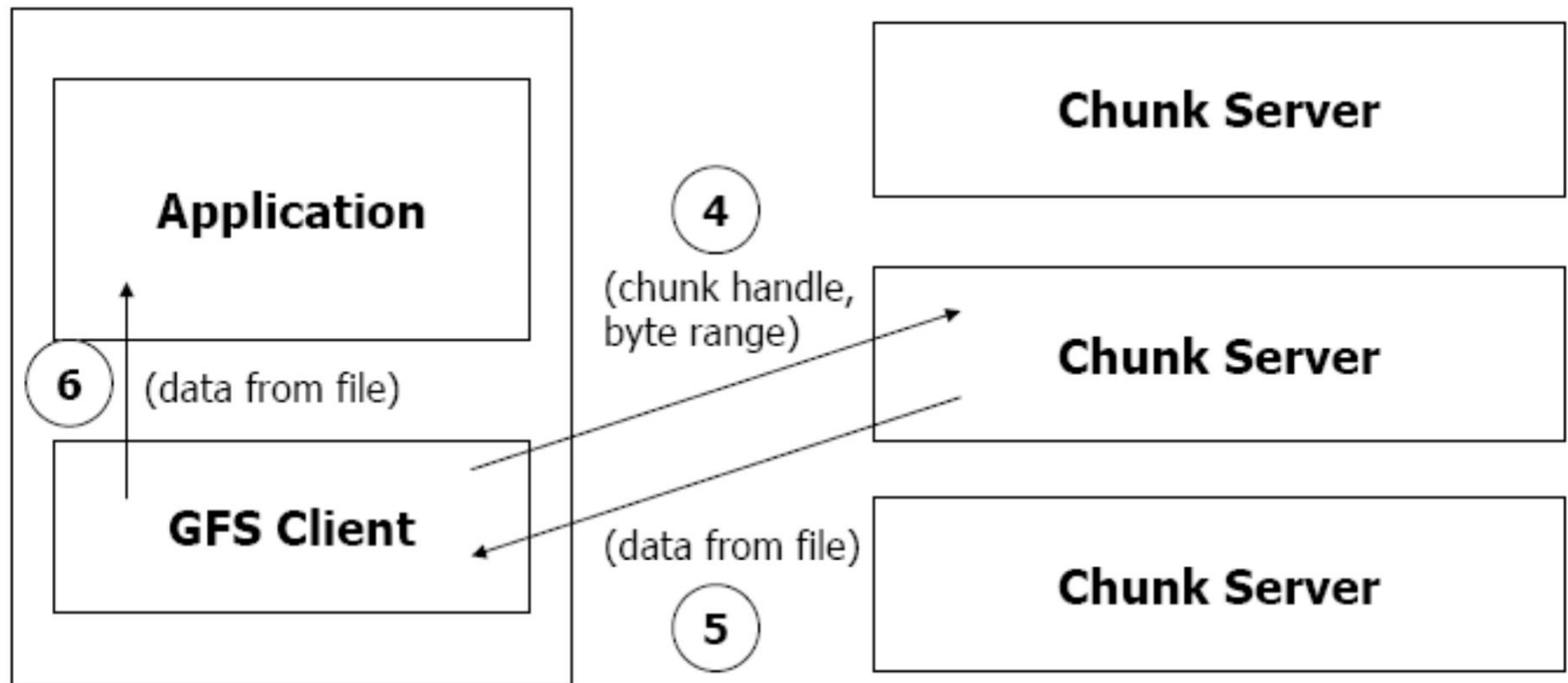
Logistics

- Extended the deadline for HW 2 to 10 PM, Friday, September 12, 2024.

GFS Operations – Read



GFS Operations – Read



GFS Operations – Updates

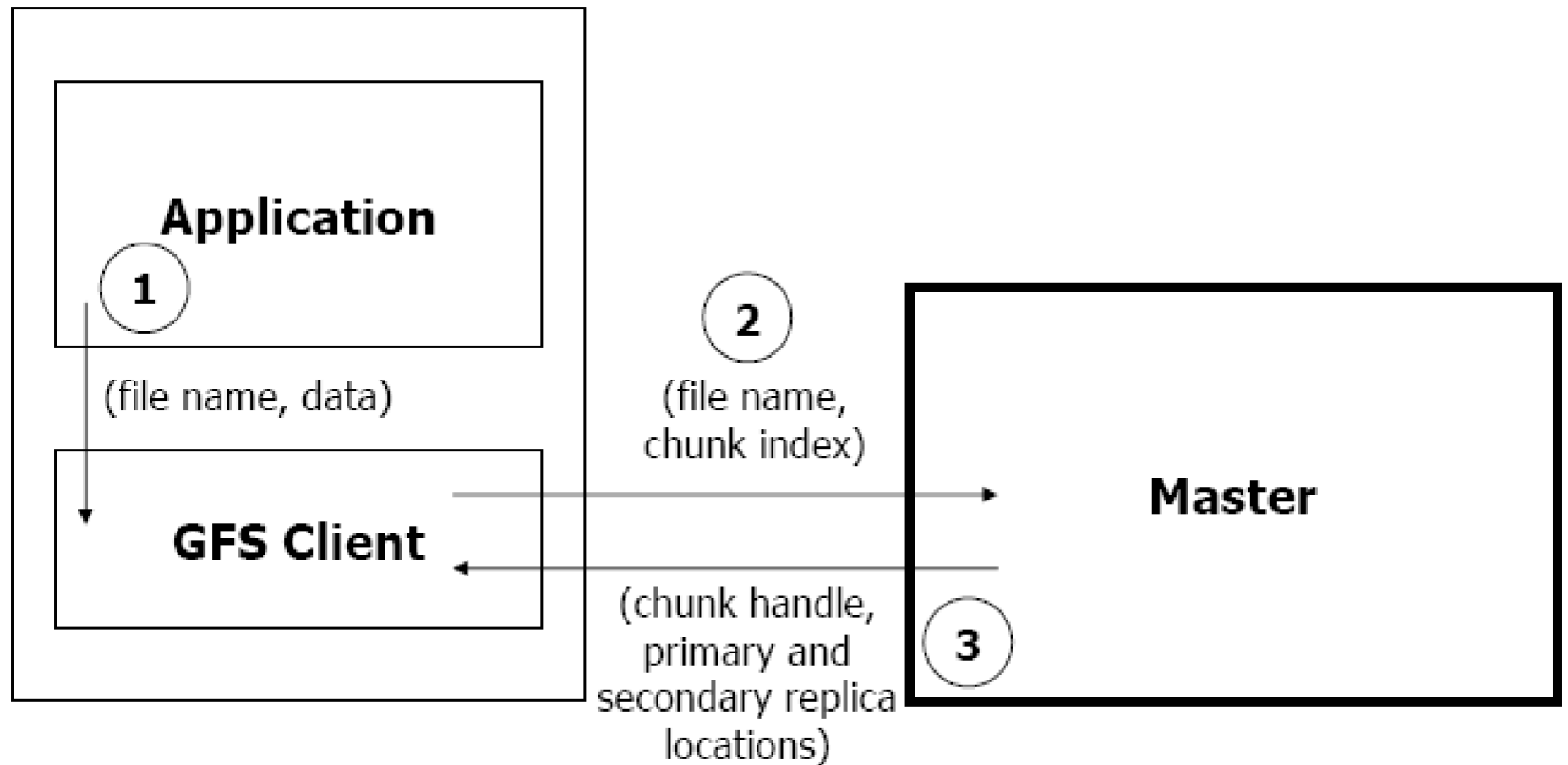
- Two update operations supported
 - **Write** – to a specific location in a file
 - **RecordAppend** – to the end of the file
- For consistency, updates to each chunk must be **ordered in the same way** at the different chunk replicas
 - Consistency means that replicas will end up with the same version of the data and not diverge
- For this reason, for each chunk, **one replica is designated as the primary**
- The other replicas are designated as **secondaries**
- Primary defines the update order
- All secondaries follows this order

Primaries

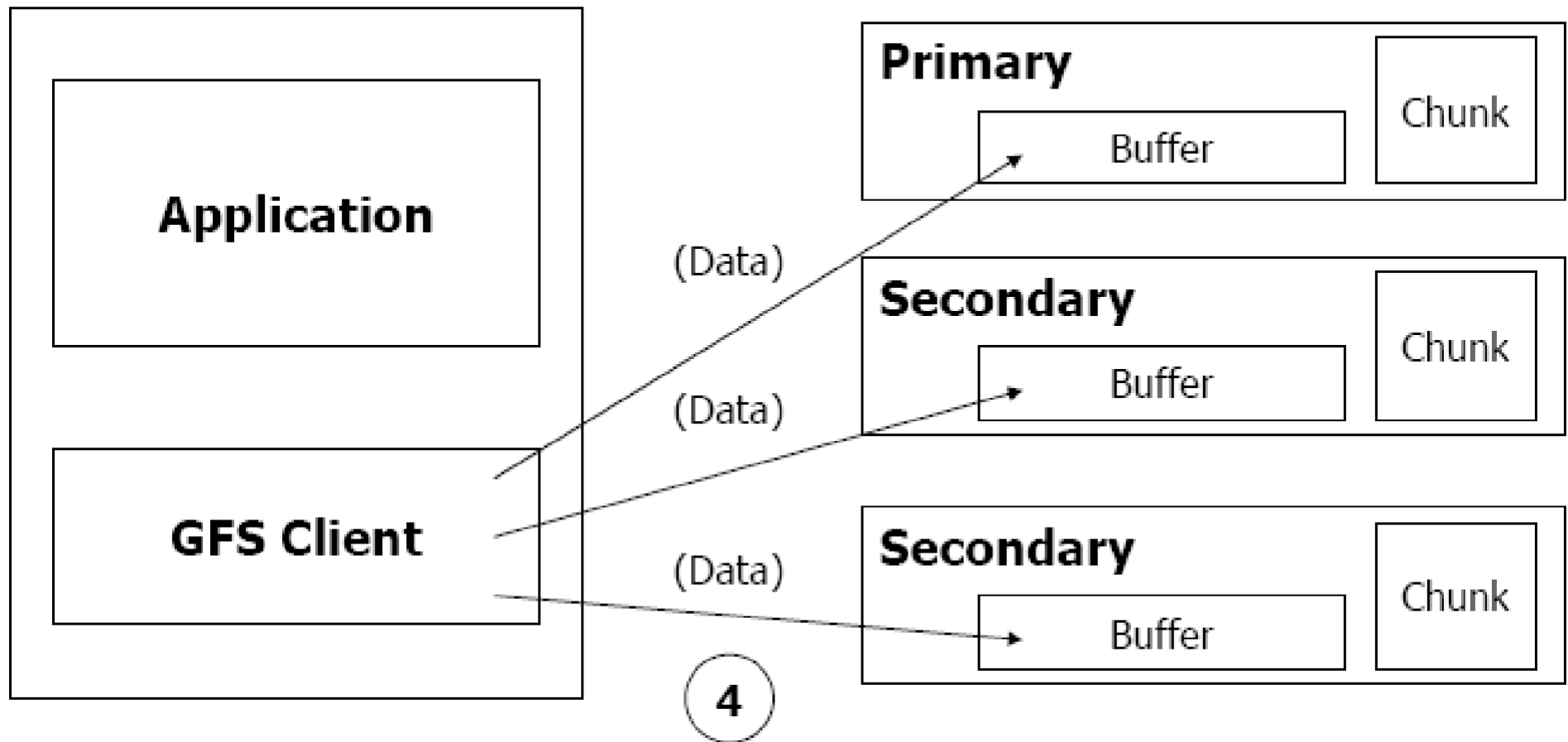
- For correctness, at any time, there needs to be one **single primary for each chunk**
 - Or else, they could order different writes in different ways
- To ensure that, GFS uses **leases**
 - Master selects a chunkserver and grants it lease for a chunk
- The chunkserver holds the lease for a period T after it gets it, and behaves as primary during this period
- The chunkserver can **refresh the lease** endlessly
- But if the chunkserver can't successfully refresh lease from master, it stops being a primary
- If master doesn't hear from primary chunkserver for a period, the master gives the lease to some other chunkserver
- So, at any time, **at most one server is the primary for each chunk**
 - But different servers can be primaries for different chunks.

Write

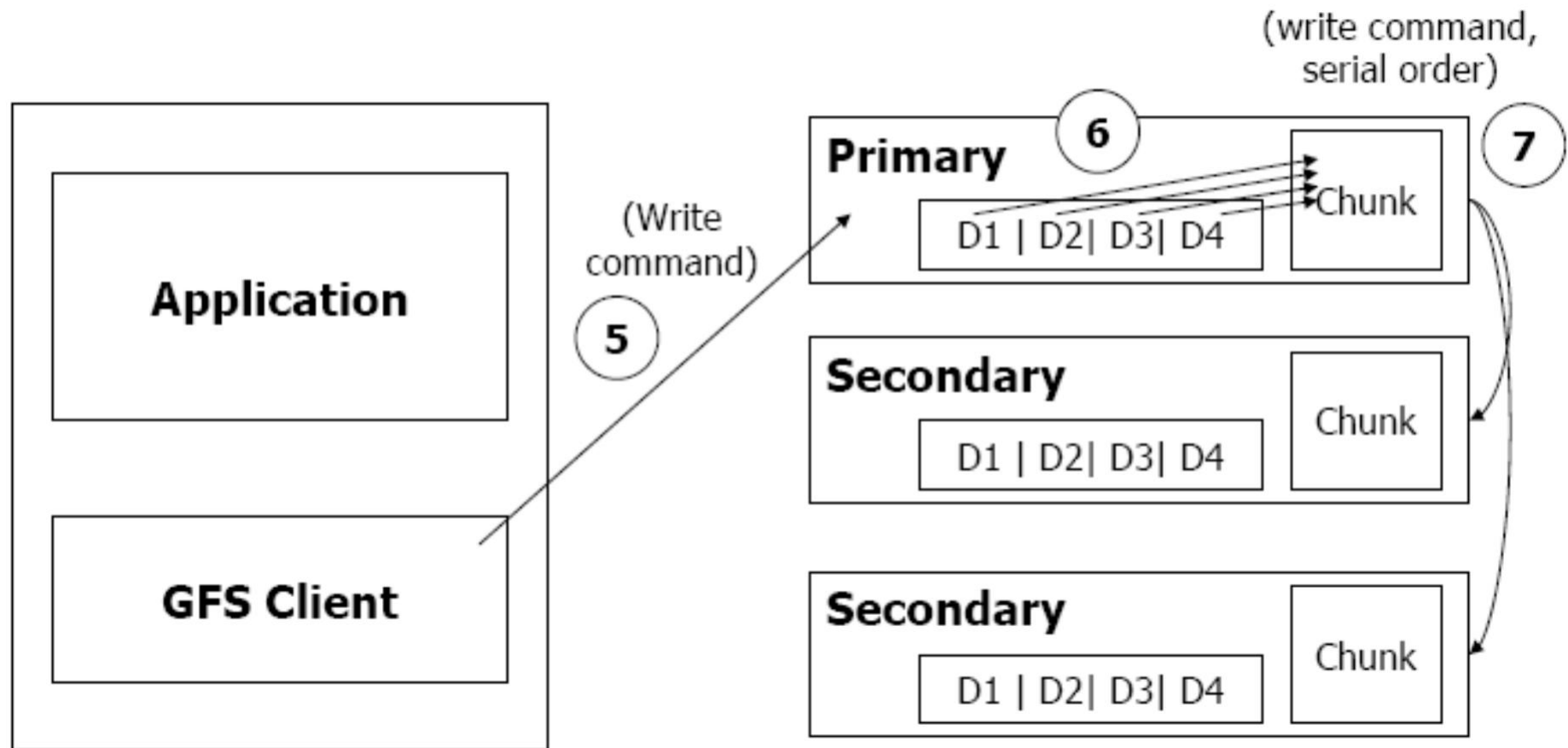
4 pages of pictures



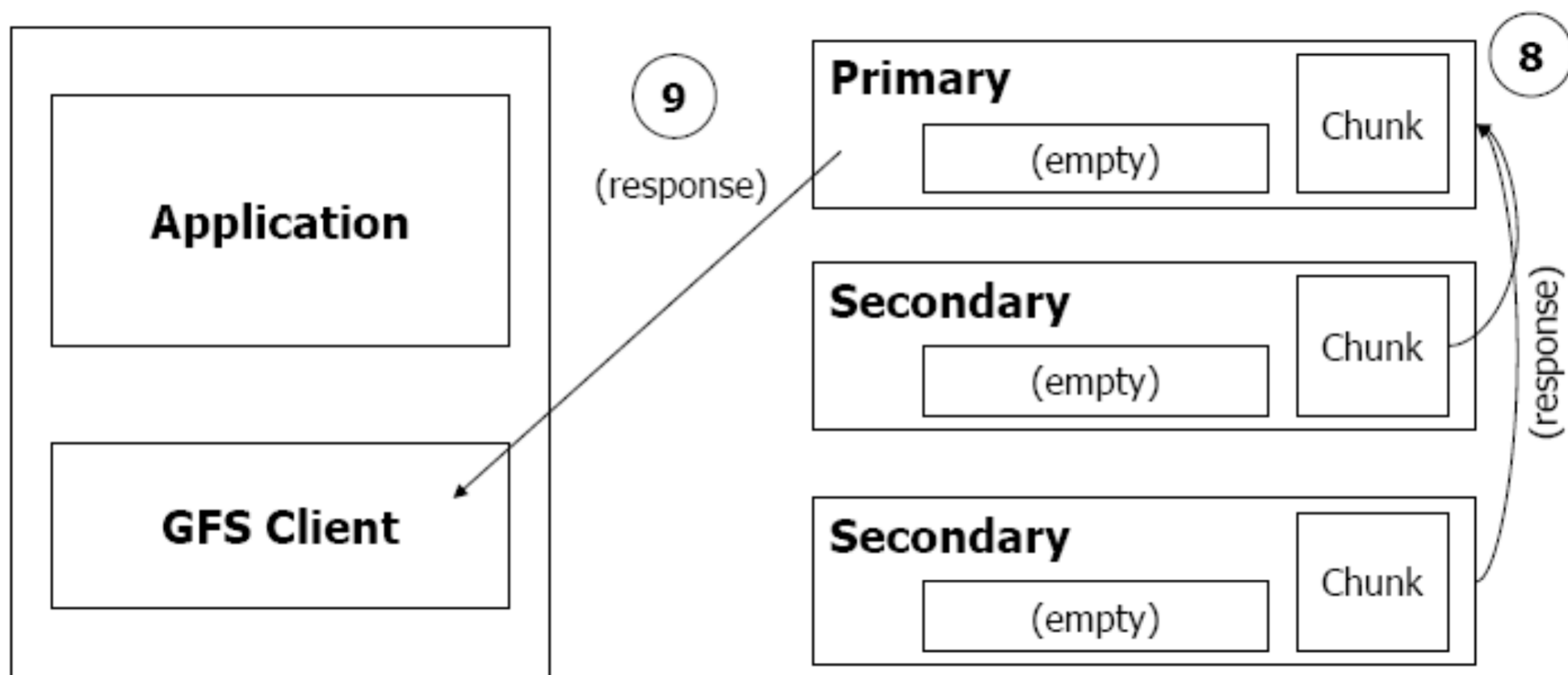
Write



Write



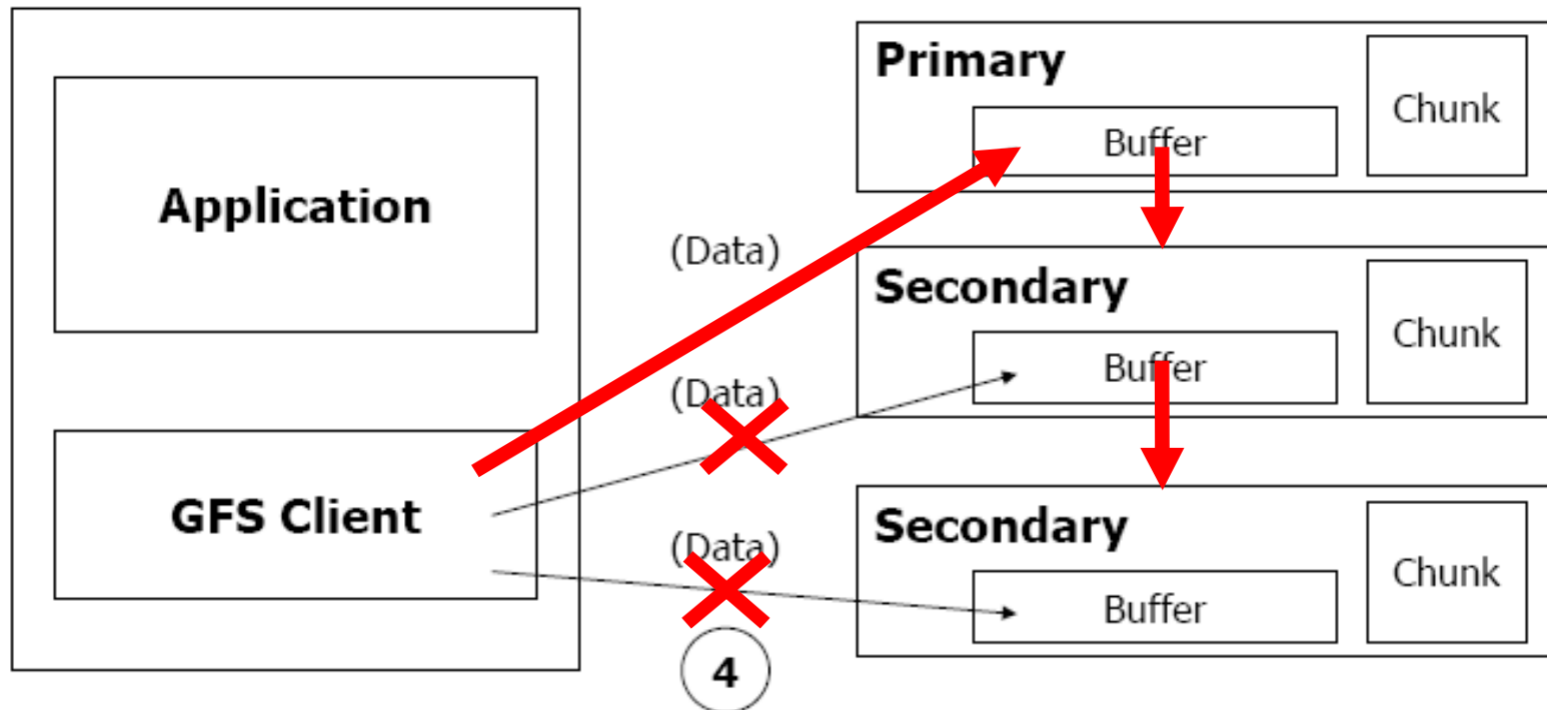
Write



Write

- Primary enforces one update order across all replicas for concurrent writes
- It also waits until a write finishes at the other replicas before it replies
- Therefore:
 - We'll have identical replicas
 - But, file region may have fragments from different clients
 - Writes to different chunks may be ordered differently by their different primary chunkservers
- Thus, writes are consistent but the impact of concurrent write is undefined in GFS

Data From Client to Chunkservers



- Refer back to the picture earlier as it shows the client sending the data to be written to each chunkserver.
- Actually, the client doesn't send the data to everyone.
- **It sends the data to one replica, then replicas send the data in a chain to all other replicas**
- Why? To maximize bandwidth and throughput!

Record Append

- The client specifies only the data, not the file offset
 - File offset is chosen by the primary
 - Why do they have this?
- Provide meaningful semantic: at least once, atomically
- Because the system is not constrained as to where to place data, it can get atomicity without sacrificing concurrency

Record Append Steps

1. Application originates a record append request.
2. GFS client translates request and sends it to master.
3. Master responds with chunk handle and (primary + secondary) replica locations.
4. Client pushes write data to all locations.
5. Primary checks if record fits in specified chunk.
6. If record does not fit, then:
 - The primary pads the chunk, tells secondaries to do the same, and informs the client of the inability/error.
 - Client then retries the append with the **next** chunk.
 - The primary chunk server for the next chunk may not be the same as this primary! So client has to retry.
7. If record fits, then the primary:
 - appends the record at some offset in chunk,
 - tells secondaries to do the same (specifies offset),
 - receives responses from secondaries,
 - and sends final response to the client.

Record Append Steps

- GFS may insert padding data in between different record append operations
- Preferred that applications use this instead of write
- Applications should use mechanisms such as **checksums** with unique IDs to handle padding

Summary of GFS

- Optimized for large files and sequential appends
 - large chunk size
- File system API tailored to stylized workload
- Single-master design to simplify coordination
 - But minimize workload on master by not involving master in large data transfers
- Implemented on top of commodity hardware
 - Unlike AFS/NFS, which for scale, require a pretty hefty server

Haystack* – The Object Store of Facebook

- Files handled by facebook include audio/video files and photographs as the platform is used primarily to share photos and audio/video content across friends.
- The file system should support quick delivery of content at planet scale.
- One possibility is to use Content Distribution Networks (CDNs).

* Beaver et al., Finding a needle in haystack: Facebook's photo storage, USENIX OSDI 2010.

CDN Vs Facebook

- There are some fundamental differences in how CDNs work and the applications that drive Facebook.
- CDNs are geared towards optimizing access for popular (frequently used) data.
- CDNs cannot handle/optimize the case of long tail.

Other Observations

- For higher throughput, it is an advantage to keep as much metadata in memory as possible.
- This requires also minimizing the size of metadata per object.
- Identify which fields of the traditional filesystem metadata can be dropped.
- Notice that most files are not shared.
 - The owner is the only one to have the ability to update.
 - All others can only read.

The Haystack System

- Three main components
 - The Haystack Store
 - has the filesystem metadata and encapsulates the persistent storage system for photos
 - The Haystack Directory
 - maintains the logical to physical mapping along with application metadata.
 - The Haystack Cache
 - Functions as an internal CDN and serves requests for hot photos.

The Haystack Store

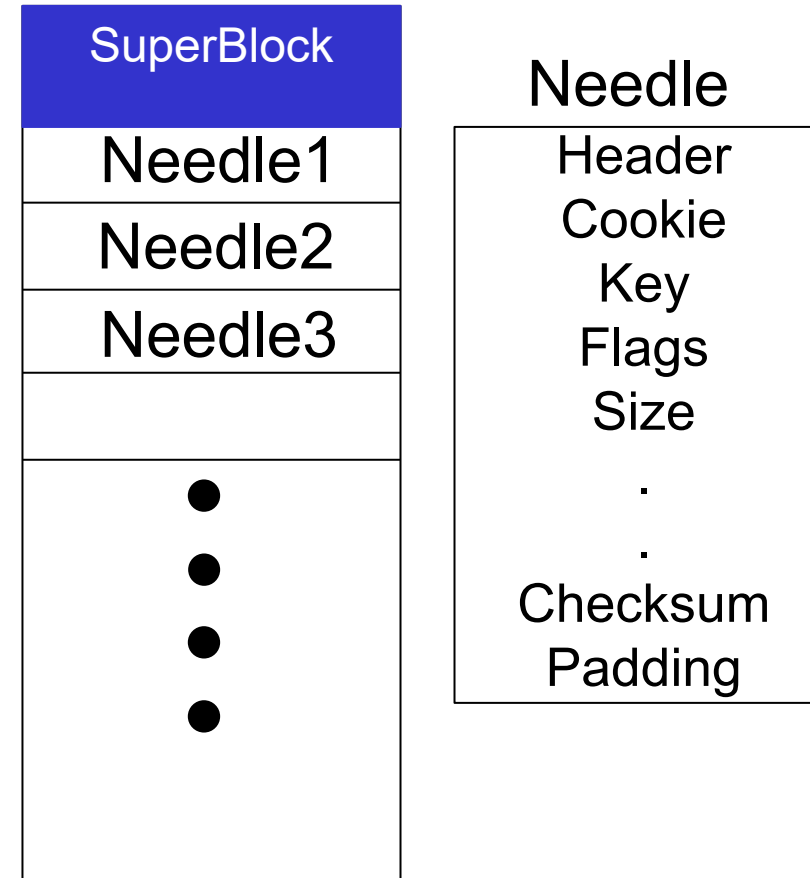
- Arranged as physical volumes with each physical volume storing millions of photos
- Multiple photos in a single file and hence the files are very large.
- Physical volumes are grouped into logical volumes.
- Haystack replicates a photo on a logical volume by writing the contents of the photo to all the physical volumes that constitute the logical volume.

The Haystack Store

- A Haystack Store machine represents a physical volume as one large file.
- The file has one superblock followed by a sequence of needles.
- Each needle represents a photo stored in Haystack.
- The Haystack Store machine accesses a photo using only the id of the corresponding logical volume and the file offset at which the photo resides in the logical volume.

The Haystack Store

- Each Store machine is a huge physical volume.
- Typically, 100s of GBs.
- The entire volume is a single file!
- The volume is therefore viewed as a superblock with a list of needles.
- Each needle represents a photo.



The Haystack Store

- To retrieve needles quickly, each Store machine maintains an **in-memory** data structure for each of its volumes.
- This data structure is a hash that maps the key to other fields of the needle.
- After a crash, each Haystack Store machine reconstructs this mapping directly from the volume file before processing requests.

The Haystack Store

- Each Haystack Store machine can access a photo quickly using only the id of the corresponding logical volume and the file offset at which the photo resides.
- The store can retrieve the filename, offset, and size for a particular photo without needing disk operations.

The Haystack Store

- Retrieving needles quickly is an essential ability of the Haystack Store.
- Each Haystack Store machine keeps open file descriptors for each physical volume that it manages.
- Also, it stores an in-memory mapping of photo ids to the filesystem metadata (i.e., file, offset and size in bytes) to be able to retrieve that photo.

The Haystack Directory

- Four main functions
 - provides a mapping from the logical volumes to the physical volumes
 - balances the load across the logical volumes and spread reads across the physical volumes.
 - determines whether a CDN or a Cache handles the request to access a given photo
 - marks certain physical volumes as read-only once these physical volumes reach their full capacity or because of operational reasons
- The system adds more physical volumes as the existing volumes reach their storage limits.
- The new volumes are marked for write. Only volumes that are marked for write can receive uploads

The Haystack Cache

- The cache keeps a mapping of the photo id to the location of the photo.
- The cache is essentially a distributed hash table.
- The cache is like an internal CDN.
- Two rules to determine if an object/photo is kept in the cache.
 - the request being served from the store comes directly from a user and not a CDN, and
 - the photo is fetched from a machine that is enabled to receive uploads.

Operations: Read/Write/Delete

- We now look at steps that Haystack uses to store a photo, serve a photo, and delete a photo.

Serving a Photo

- Consider a user requesting for a photo thorough a browser.
- The web server constructs a URL for the request.
- Each URL is of the form `http://<CDN-Name>/<Cache-Name>/<Machine-id>/<Logical Volume, Photo>`.
- Observe the hierarchical nature of the URL.

Serving a Photo

- The first part of the URL specifies the CDN that can possibly serve the request
- If the request is not satisfied at the specified CDN, the CDN can remove the corresponding part of the URL and forward the request to the Haystack Cache specified in the URL.
- If the Cache fails to serve the request, then the Cache can remove the corresponding part of the URL and forward the request to the machine (Haystack Store) that can serve the request.
- At this level, if the photo is found in the specified volume at the given offset, it is served.

Serving a Photo

- The request to the Store is accompanied with the logical volume id, key, and the cookie fields.
- These attributes are part of the needle data fields.
- The cookie field, being a random number assigned at the time of upload and is embedded in the URL, prevents guessing attacks that can prepare valid URLs to mount denial-of-service attacks.
- The Haystack Store reads the volume from the provided offset, makes sure that the photo is not (marked) deleted, verifies the cookie, and then passes the photo to the Haystack Cache.

Storing a Photo

- When uploading a photo into Haystack web servers provide attributes such as the logical volume id, key, alternate key, cookie, and data to the Haystack Store machines.
- Each machine then synchronously appends needle images to its physical volume files and updates in-memory mappings as needed.
- Each photo is stored in four different sizes with the same key.
 - Helps address current realities such as multiple device form factors

Storing a Photo

- Operations on photos such as rotations create a new needle.
- This new needle will have the same key and alternate key as the original photo.
- There are now two possibilities.
 - The new needle is stored in the same logical volume.
 - In this case, the new needle is appended to the same physical volumes as the original needle.
 - The Haystack store uses the rule that the needle with a larger offset is considered as a later version.
 - The new needle is stored in a different logical volume
 - The Haystack Directory updates the metadata entries corresponding to the original needle.

Delete a Photo

- Deleting a photo is a logical operation.
- The Haystack Store machine that has the photo sets the delete flag in both the in-memory mapping and synchronously in the volume file.
- Since requests to retrieve photos first check the in-memory flag, such photos are never returned as the answer to the request.
- As with any logical delete models, the space held by deleted photos is reclaimed via a future internal operation.
- This is called **compaction**. Periodically,
 - each Haystack Store machine goes through the entire set of needles,
 - copies the valid needles to a new file, and
 - at the end of this process sets the in-memory structures to the new file.

Other Features

- The size of the metadata per photo stored in Haystack is about 40 Bytes.
 - Compare with ~500 Bytes possibly for the Linux `xfs_inode_t` data structure.
- Haystack is designed with fault-tolerance in mind.
 - Replication
 - Mechanisms such as heartbeat for identifying drives with errors

Other Popular Systems

- Google Colossus
 - More like the next version of GFS
 - Designed for much more scale.
 - Smaller block size, more metadata, metadata no longer in memory but in database!
 - Multiple Master nodes
 - Data marked as hot vs cold.
 - Hot data is used more frequently at the given time instant
 - Photos, news, etc.
 - Cold data is old data that is not used heavily.
 - Each disk to have a mix of hot and cold data.
 - Otherwise, can run into overuse/underuse.

Other Popular Systems

- The Facebook Tectonic file system
 - Generalizes from Haystack. Haystack meant mainly for blob storage.
 - Tectonic supports multiple application scenarios.